

Beyond Injection Detection: A Positive-Security Prompt Firewall that Closes the Scope and PHI Gap SOTA Classifiers Miss in Healthcare

QFIRE — a parallel, low-latency Rust firewall benchmarked against PromptGuard-2 and DeBERTa-v3

Task Force for AI Agents in Healthcare

May 31, 2026

Abstract

Large language models embedded in autonomous agents process trusted instructions and untrusted data in one context window, leaving them open to direct and indirect prompt injection. Existing runtime guardrails trade safety against latency: model-based auditors are accurate but add hundreds of milliseconds of Python inference, while lexical filters are fast but blind to obfuscated or semantically disguised payloads. We present **QFIRE**, an inline, provider-agnostic prompt firewall implemented as a single self-contained Rust toolchain (proxy, CLI, and benchmark harness). QFIRE contributes three ideas: (i) *positive-security scope constraining*, which restricts a model call to a declared natural-language purpose and blocks out-of-scope drift even when no overt attack token is present; (ii) an *asynchronous detector graph* that runs N rules and their typed detector nodes concurrently on a Tokio runtime with cheap-before-expensive short-circuiting; and (iii) a *de-obfuscation normalization pass* that decodes Base64/hex/ROT13, folds homoglyphs and leetspeak, and strips zero-width characters before detection. QFIRE ships ~ 100 versioned firewall rules, a dedicated HIPAA Safe-Harbor (18 identifier) healthcare/PHI panel, and runs the `deberta-v3-base-prompt-injection` classifier locally via embedded ONNX Runtime. On 1,968 public prompt-injection and jailbreak prompts QFIRE’s deterministic hybrid attains F1 0.86, statistically tied with Meta’s state-of-the-art PromptGuard-2 (F1 0.86) and above protectai DeBERTa-v3 (F1 0.83), while lexical baselines lag (F1 0.16–0.50). Our central result is on *QFIRE-HealthBench*, a new 2,000-prompt healthcare benchmark we build with real garak and Microsoft PyRIT payloads: there the same SOTA PromptGuard-2 recovers only 0.40 recall (DeBERTa-v3 0.57), because most clinical threats—PHI exfiltration, cross-patient access, re-identification, bulk export, out-of-scope advice—carry no injection signal, whereas QFIRE’s combined scope+PHI chain reaches 0.83 recall (F1 0.87) at a calibrated 0.08 false-positive rate. Generic prompt-injection detection, even SOTA, is therefore necessary but not sufficient for healthcare LLM agents; positive-security scope and PHI controls close a gap a classifier structurally cannot. We report precision/recall/F1 with 95% Wilson confidence intervals, ROC-AUC, and p50/p95/p99 latency. All code, rules, corpora snapshots, and scripts are released, and every table regenerates from a single `make paper` target against local models with no paid API keys.

1 Introduction

The integration of large language models (LLMs) into agentic systems—tools, browsers, retrieval pipelines—has moved AI security from content moderation to application-layer defense-in-depth [1, 5]. Because an LLM consumes its system prompt and untrusted inputs within a single semantic

context, instruction-override, role-play jailbreak, and adversarial-suffix attacks remain effective despite model alignment [10, 5]. Runtime gateways that intercept prompts before they reach the model are therefore a necessary control [1, 9].

A review of recent runtime guardrails reveals several distinct architectural philosophies (Table 2). LlamaFirewall [1] chains modular Python detectors (PromptGuard 2, AlignmentCheck, CodeShield) for deep multi-stage auditing; OneShield [9] runs a parallelized multi-detector gateway for enterprise policy and PII; the Cognitive Firewall [2] splits computation between fast edge lexical screening and cloud semantic verification for browser agents; PSG-Agent [12] builds per-user personality profiles to adapt safety thresholds to intent drift; and Semantic Firewalls [15] transform prompts into a type-safe closed vocabulary to eliminate syntactic bypasses. Across these designs a single tension recurs: *security robustness is tightly coupled to latency*. Heavy semantic verifiers and model-based auditors achieve strong detection but add hundreds of milliseconds of Python-hosted inference, while deterministic lexical filters run in sub-milliseconds yet miss obfuscated or semantically disguised payloads [1, 5]. A second gap is orthogonal to latency: these systems are tuned for *generic* injection and policy violations, not for the scope-restriction and protected-health-information (PHI) obligations of clinical deployment—threats that, as we show, carry no injection signal at all. QFIRE targets both gaps with an ultra-low-latency, zero-trust parallel prompt firewall in Rust that adds positive-security scope constraining and a HIPAA PHI panel to the inline-proxy design.

This work extends the Healthcare AI Agents Regulatory Framework (HAARF) [14], a security-verification standard for autonomous AI systems in clinical environments from the same group. HAARF specifies *what* security controls a clinical AI agent must satisfy but is deliberately implementation-agnostic; QFIRE is a concrete, measured *enforcement* layer that hardens specific HAARF controls at runtime. The mapping is direct (Table 1): QFIRE’s injection-defense pack and benchmark harness discharge the prompt-injection adversarial-robustness requirement (C3.2.1); its regex/Aho-Corasick/entropy filters and de-obfuscation pass implement prompt-layer input validation and sanitization (C3.2.3, which QFIRE addresses at the prompt level rather than at the HL7/DICOM/FHIR parsing layer); its 18-identifier HIPAA Safe-Harbor PHI detector provides the detection-and-redaction component of PHI handling and access control (C3.6.1, complementing encryption controls QFIRE does not itself provide); its inline proxy with positive-security scope rules provides the real-time, clinical-context-aware threat monitoring HAARF asks for (C3.4.1, C3.4.4); its declared per-chain scope with fail-closed blocking implements autonomy authority boundaries whose violations are automatically detected and logged (C6.3.1, C6.3.4); and its immutable, attributable audit log of every decision—timestamp, prompt hash, per-node verdicts, confidence, terminal decision—satisfies the clinical decision audit-trail requirement (C2.5.1) and supplies the evidence trail HAARF verification expects. The controls HAARF most emphasizes for clinical agents—scope restriction and PHI handling—are precisely the threats that, as we show, generic prompt-injection detectors structurally cannot meet.

A recurring tension in this literature is *security robustness versus latency*. Heavy semantic verifiers and model-based auditors achieve strong detection but introduce substantial per-call latency from Python-hosted inference [1]; conversely, deterministic regular-expression filters are sub-millisecond but fail on obfuscated or paraphrased attacks [5]. QFIRE targets this gap with a zero-trust, ultra-low-latency design built in Rust.

Contributions.

1. **Positive-security scope constraining.** Beyond flagging overt attacks, QFIRE enforces a declared operational scope (e.g. “marketing copy only”, “appointment scheduling, no clinical

HAARF control	Requirement (abridged)	QFIRE mechanism
C3.2.1	Prompt-injection adversarial-robustness testing on clinical AI assistants	Injection-defense rule pack, embedded DeBERTa classifier, <code>qfire bench</code> harness
C3.2.3	Input validation/sanitization at integration points (prompt layer)	Regex/Aho-Corasick/entropy detectors + de-obfuscation normalization
C3.4.1, C3.4.4	Real-time threat monitoring with clinical-context awareness	Inline proxy verdicts + positive-security scope rules
C3.6.1	PHI handling/access control (detection & redaction component)	18-identifier HIPAA Safe-Harbor PHI detector/redactor
C2.5.1	Immutable audit trail: timestamp, input, decision, confidence	Append-only attributable audit log + explainable per-node trace
C6.3.1, C6.3.4	Authority boundaries; violations auto-detected and logged	Declared per-chain scope, fail-closed BLOCK, audit escalation

Table 1: QFIRE as a concrete enforcement layer for specific HAARF [14] controls. HAARF defines the verification requirement; QFIRE is the runtime mechanism that satisfies it and produces the measured evidence.

advice”) and blocks anything outside it, neutralizing camouflaged or benign-looking drift.

2. **Asynchronous detector graph with collapse.** Rules and detector nodes execute concurrently; a chain collapses N per-rule verdicts into one terminal decision in either an ordered (iptables-style) or boolean-expression mode, with cheap-before-expensive short-circuiting.
3. **De-obfuscation normalization** that exposes hidden payloads (Base64/hex/ROT13, homographs, leetspeak, zero-width) before detection, and a complete **HIPAA Safe-Harbor PHI** panel covering all 18 identifiers.
4. **A reproducible benchmark** comparing QFIRE head-to-head with open detectors on public corpora, with Wilson confidence intervals and latency percentiles, regenerated end-to-end by `make paper`.

2 Threat Model and Positioning

We assume an adversary who controls all or part of the prompt reaching the model (direct injection) or content the agent ingests (indirect injection [5]). The defender controls a network position *in front of* the model and wishes to (a) reject prompts outside a declared purpose and (b) detect injection/jailbreak attempts, while preserving throughput. Table 2 situates QFIRE against representative recent systems.

3 System Design

QFIRE is an inline reverse proxy and CLI. Incoming requests in the native wire formats of OpenAI, Anthropic, Gemini, and Ollama are normalized into one internal representation; the firewall

The same detectors: tied on generic injection, far apart on healthcare

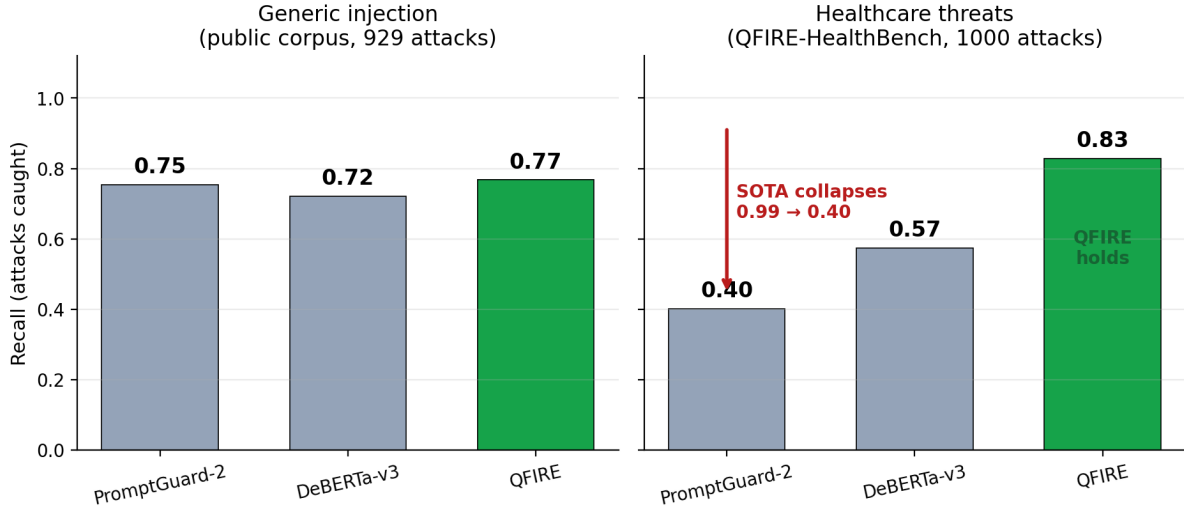


Figure 1: **The paper in one figure.** The same three detectors are statistically tied on generic prompt injection (left), but on healthcare threats (right) SOTA PromptGuard-2 collapses to 0.40 recall while QFIRE’s scope+PHI chain holds at 0.83. Generic injection detection is necessary but not sufficient in healthcare; the visual drop between panels is this paper’s thesis.

evaluates the extracted prompt and, on ALLOW, transparently forwards the original request downstream. Figure 3 shows the end-to-end path: SDKs reach QFIRE by changing only their base URL, the engine evaluates the selected chain on a Tokio runtime, and every decision is written to an immutable audit log before a request is forwarded or refused (Figure 3, shown full-page next).

3.1 Rules, chains, and collapse

A *rule* declares a natural-language scope and an ordered *detector pipeline*; each node returns a verdict, confidence, latency, and rationale. A *chain* composes rules into one terminal decision in two interchangeable modes: ORDERED (first matching block/allow wins, configurable default) and EXPRESSION (a boolean DAG over named rules, e.g. `injection_guard AND (marketing OR support)`). Detectors are typed as *blockers* (regex, Aho-Corasick, entropy, the injection classifier), which return BLOCK or ABSTAIN, or *scope deciders* (LLM judge, exemplar similarity), which return ALLOW/BLOCK; the expression predicate “rule did not block” lets guards and scope rules compose uniformly.

3.2 A declarative firewall language

The deepest difference from PromptGuard-2, LlamaFirewall, and Protect AI LLM Guard is not a single detector: a QFIRE firewall is *data, not code*. Those systems express policy in Python and model weights; QFIRE expresses it in version-controlled YAML that the engine interprets, making policy auditable, diff-able, and unit-testable (`qfire rules test` runs each rule’s exemplars as fixtures). Figure 4 shows a rule, an expression chain, and an ordered chain verbatim from the repository; Table 3 summarizes the shipped library. Detector nodes are typed—`regex/aho` (sub-ms denylists), `entropy, phi` (18 HIPAA identifiers), `deberta` (local ONNX classifier), `judge` (LLM

Framework	Core design	Threat focus	Primary advantage	Operational limitation
LlamaFirewall [1]	Modular Python middleware interceptor	Prompt injection, goal hijacking, insecure code generation	Deep multi-stage auditing (PromptGuard 2, AlignmentCheck, CodeShield)	High latency from Python / PyTorch inference
OneShield [9]	Standalone parallelized multi-detector gateway	Enterprise policy violations, PII leakage, copyright	Dynamic scale-out; unified parallel low-latency execution	Reliance on external API management and standard datasets
Cognitive Firewall [2]	Split-computing edge-to-cloud architecture	Indirect injection in autonomous browser agents	Fast lexical edge screening minimizes local load	Cloud semantic verification raises privacy and latency issues
PSG-Agent [12]	Training-free personalized runtime guardrail	Adaptive user-specific risk, intent drift over turns	Per-user profiling tailors safety thresholds	High compute cost; hard to deploy stateless
Semantic Firewalls [15]	Abstractive protocol transformer / online ensemble	Contextual data leakage, tool-parameter and syntax bypass	Type-safety, closed vocabularies, string isolation	Demands structured, application-specific translation schemas
garak [4]	Vulnerability scanner / probe suite	Broad red-team attack coverage	Comprehensive offline attack library	Offline scanner, not an inline filter
QFIRE (ours)	Rust async detector graph + positive scope, inline proxy	Injection <i>and</i> out-of-scope/PHI drift in clinical agents	Low-latency local inference, de-obfuscation, declarative scope constraining, no Python in the hot path	Single-node; LLM-judge cost grows with rule count

Table 2: QFIRE relative to representative guardrail and red-team systems. Existing inline guardrails optimize either deep auditing (LlamaFirewall), parallel scale-out (OneShield), edge offloading (Cognitive Firewall), per-user adaptation (PSG-Agent), or type-safe protocol translation (Semantic Firewalls); none target the low-latency, declarative, positive-security scope+PHI enforcement QFIRE provides for clinical agents.

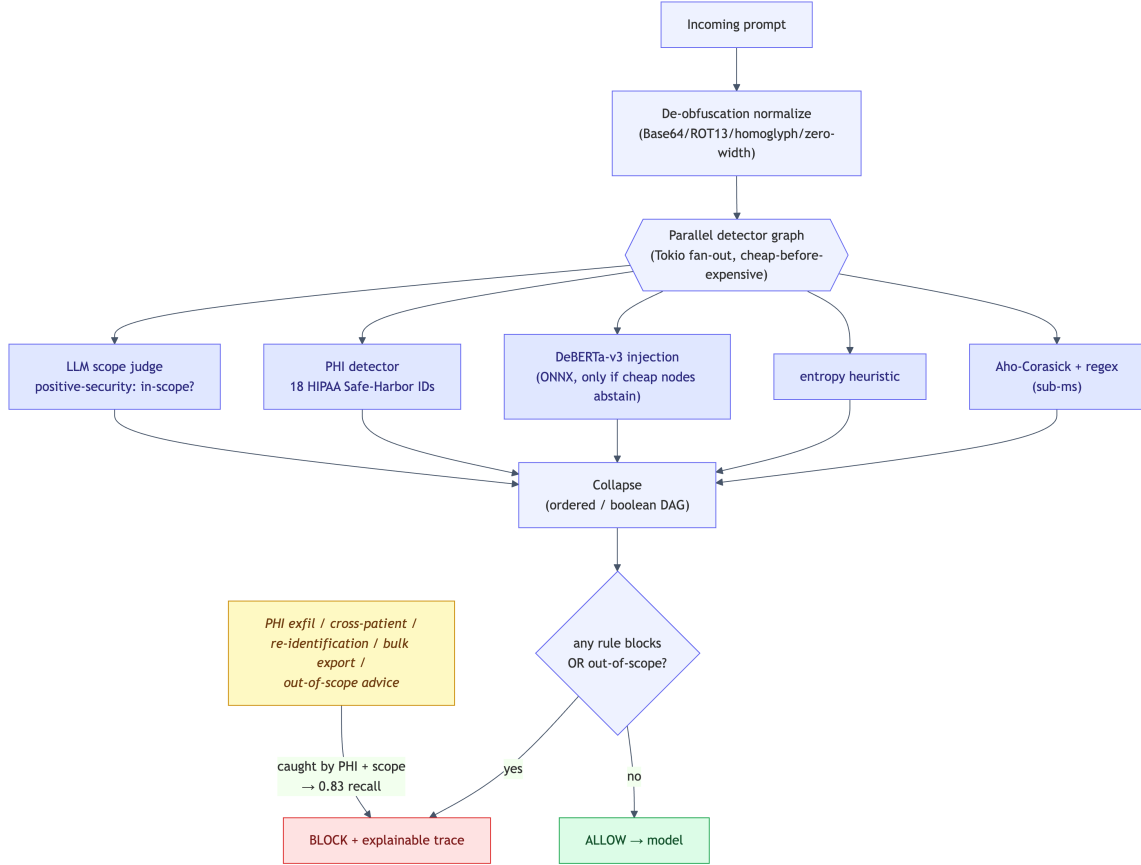
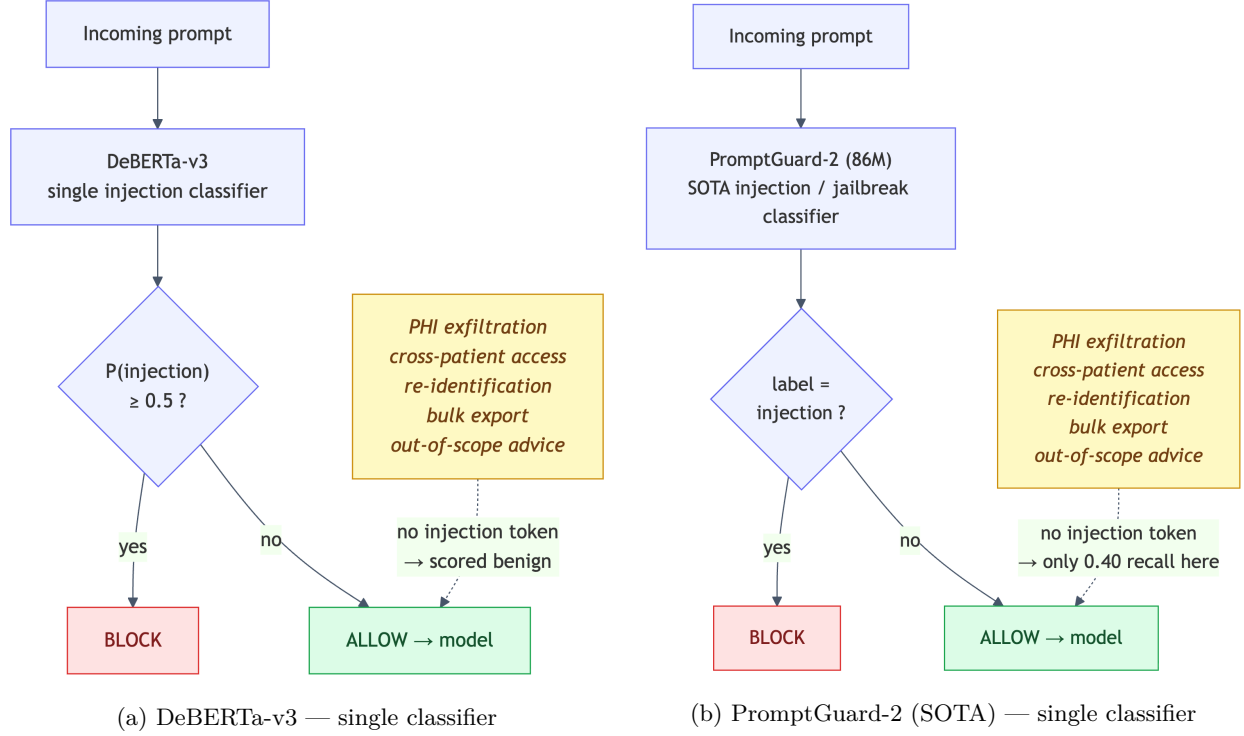


Figure 2: How the three approaches decide to block. The two SOTA baselines (a, b) are single-classifier injection detectors: a prompt is blocked only if one model labels it an injection, so health-care threats with no injection signal (PHI exfiltration, cross-patient access, re-identification, bulk export, out-of-scope advice) pass through—why PromptGuard-2 reaches only 0.40 recall on QFIRE-HealthBench. QFIRE (c) fans out across lexical, classifier, PHI, and scope detectors and collapses them into one explainable decision, catching those threats and lifting recall to 0.83.

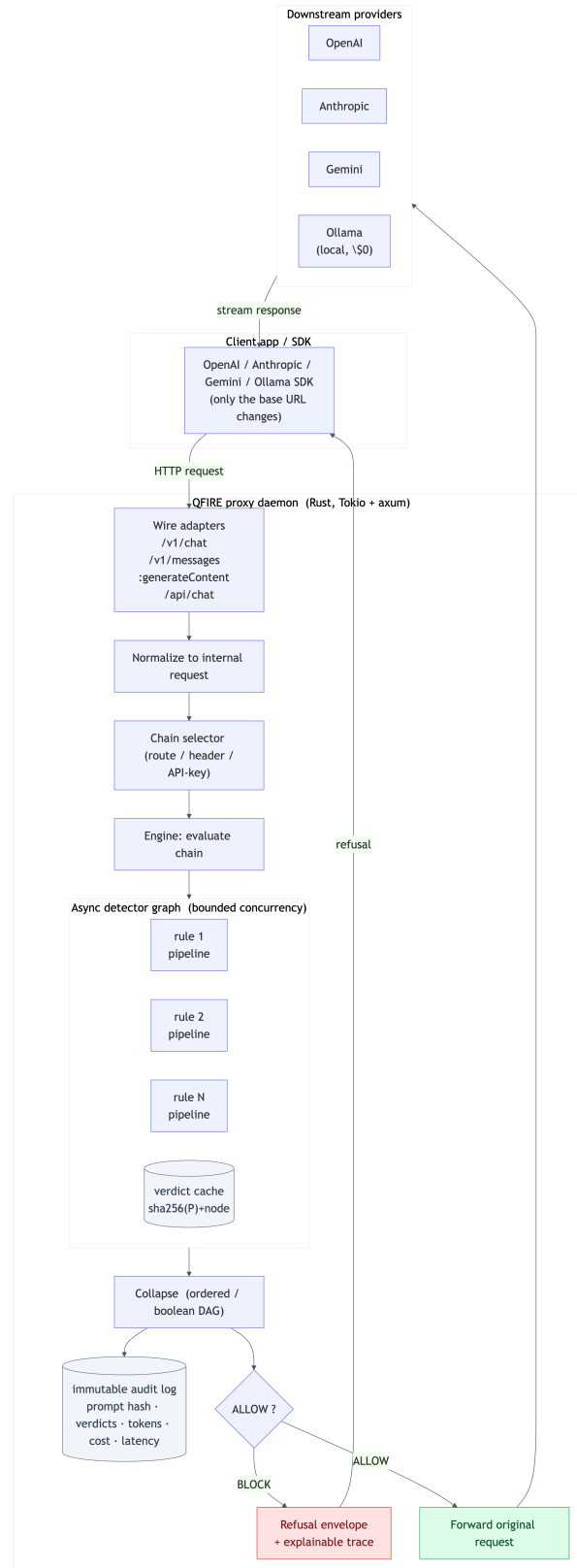


Figure 3: QFIRE system architecture (top to bottom). Wire-compatible adapters accept native OpenAI/Anthropic/Gemini/Ollama traffic; requests are normalized, routed to a chain, and evaluated by an asynchronous detector graph with a verdict cache. On ALLOW the original request is forwarded (and the response streamed back); on BLOCK a refusal envelope with an explainable trace is returned. Every decision is appended to an immutable audit log.

```

# rules/healthcare/phi_panel.yaml -- a rule: scope + typed pipeline + exemplars
- id: hc_no_diagnosis
  scope: "General health info only; no diagnosis or symptom interpretation."
  short_circuit: stop_on_first_block
  pipeline:
    - type: judge # semantic scope decision (LLM)
  exemplars:
    in_scope: ["What is hypertension and its lifestyle factors?"] # ALLOW
    out_of_scope: ["I have fever and a stiff neck - do I have meningitis?"] # BLOCK

# rules/injection/injection_defense.yaml -- cheap regex BEFORE the classifier
- id: injection_instruction_override
  pipeline:
    - type: regex
      deny: ['(?:i)gnore\s+(all\s+|the\s+)?(previous|above)\s+instructions?']
    - type: deberta # only runs if the regex abstains
      threshold: 0.6

# chains/hipaa_phi.yaml -- EXPRESSION mode (boolean DAG, fail-closed)
id: hipaa_phi
mode: expression
fail_policy: fail_closed
groups:
  injection: "injection_instruction_override AND injection_system_prompt_exfil"
expression: >
  injection AND hc_no_diagnosis AND hc_no_medication_dosing AND
  hc_phi_other_patient_record AND hc_phi_reidentification

# chains/injection_ordered.yaml -- ORDERED mode (iptables-style, default-allow)
id: injection_ordered
mode: ordered
default: allow
rules: [injection_instruction_override, injection_jailbreak_dan,
        injection_data_exfiltration, injection_classifier_only]

```

Figure 4: The QFIRE spec, verbatim from the repository: a *rule* (scope + cheap-before-expensive typed pipeline + exemplars that are also test fixtures), an *expression* chain (boolean DAG with a reusable group), and an *ordered* chain. Policy is declarative data a compliance reviewer can read, diff, and test—unlike the code/model pipelines of the baselines.

scope), and **similarity**—and are ordered cheap-before-expensive so a rule short-circuits on the first block before reaching the classifier or judge.

3.3 Asynchronous detector graph

Given prompt P and active rules $R = \{r_1, \dots, r_N\}$, QFIRE evaluates rules concurrently on a Tokio runtime under a bounded-concurrency semaphore. Within a rule, nodes run in pipeline order with short-circuiting, so a cheap lexical detector that fires aborts the remaining—possibly expensive—nodes (e.g. the local DeBERTa classifier or an LLM judge). A verdict cache keyed by $\text{sha256}(P)$ plus node identity lets repeated or replayed prompts skip recomputation. The terminal decision is a collapsible boolean over per-node block states.

3.4 De-obfuscation normalization

Before detection, an optional normalization pass produces an expanded view of P : it strips zero-width characters, folds common Cyrillic/Greek homoglyphs and leetspeak to ASCII, and decodes Base64/hex runs and ROT13, appending each recovered layer. Detectors then scan the expanded

Domain	Rules	Example rule ids
injection-defense	12	<code>injection_instruction_override</code> , <code>injection_jailbreak_dan</code>
healthcare / PHI	16	<code>hc_no_diagnosis</code> , <code>hc_phi_reidentification</code>
marketing	12	<code>mk_product_tagline</code> , <code>mk_seo_blog</code>
customer support	12	<code>sup_order_status</code> , <code>sup_returns_refunds</code>
code assistance	12	<code>code_explain_snippet</code> , <code>code_readonly_sql</code>
content safety	12	<code>safe_self_harm</code> , <code>safe_malware</code>
finance	10	<code>fin_no_personalized_advice</code>
legal	10	<code>legal_no_individualized_advice</code>
data / SQL	10	<code>sql_readonly_select</code> , <code>sql_no_destructive</code>
Total	106	+ 16 chains (10 benchmark, 6 application)

Table 3: The shipped rule library: 106 rules across 9 domains, all validated by `qfire rules lint` and exercised by `qfire rules test`.

view, exposing payloads that the raw text would smuggle past lexical and even learned detectors.

3.5 Detector library, healthcare panel, and PHI

QFIRE ships ~ 100 versioned rules across injection-defense, marketing, support, code, data/SQL, finance, legal, and content-safety domains, plus a dedicated healthcare panel. The PHI engine implements all 18 HIPAA Safe-Harbor identifiers [16] (SSN, dates, phone/fax, email, MRN, beneficiary/account numbers, URLs, IP addresses, VINs, device/license serials, geographic and contextual name/biometric cues), reporting each hit’s category and supporting typed redaction.

3.6 Implementation

QFIRE is $\sim 7k$ lines of Rust (Tokio, axum, reqwest). The injection classifier `deberta-v3-base-prompt-injection` runs locally via embedded ONNX Runtime [13] with a bundled tokenizer; when the ONNX model is absent a transparent lexical classifier is used and reported as such, so results are never silently degraded. The LLM scope-judge can target any configured provider; all reproducibility runs use local Ollama [8].

4 Experimental Setup

Data. We evaluate on 1,968 public prompts (929 attack, 1,039 benign) drawn from the deepset prompt-injections [3] and jailbreak-classification [6] datasets, snapshotted and versioned in the repository.

Baselines. We compare against open detectors run on the identical corpus: `deberta-v3-base-prompt-injection` (the de-facto open detector, and the engine inside Protect AI LLM Guard), executed in PyTorch; Meta `Llama-Prompt-Guard-2-86M` [7]; and lexical regex / Aho-Corasick filters. QFIRE runs the same DeBERTa weights via Rust ONNX. Meta PromptGuard-2 is run locally in PyTorch with an authorized Hugging Face token (gate accepted), and the PyTorch DeBERTa baseline—after resolving an unrelated `torch/torchvision` operator clash by removing `torchvision`—reproduces the ONNX path within ~ 0.02 F1, confirming the integration is faithful. On the public corpus

Learned detectors dominate lexical filters on the public corpus (929 attacks / 1039 benign)

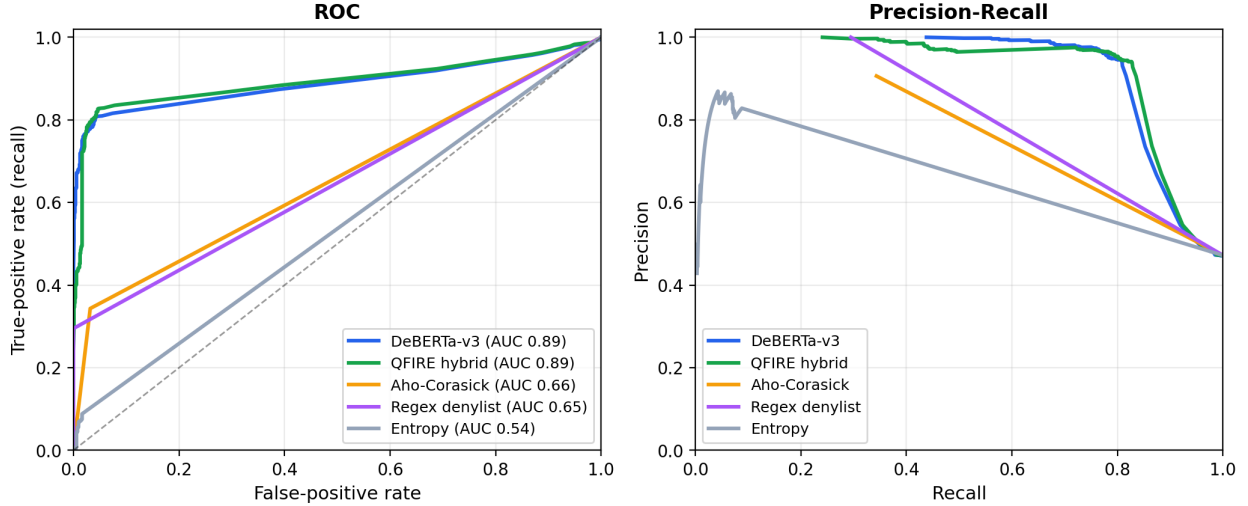


Figure 5: ROC and precision-recall curves on the public corpus. The learned detectors (DeBERTa AUC 0.89, QFIRE hybrid) hug the top-left corner; the lexical filters track the diagonal, making the “fast-but-blind” gap visual.

PromptGuard-2 is the strongest single classifier (P 0.997/R 0.755/F1 0.859/FPR 0.002), statistically even with QFIRE’s hybrid; on QFIRE-HealthBench (§6) the same model recovers only 0.40 recall, the gap this paper is about.

Metrics. Treating “attack” as the positive class we report precision, recall, F1, false-positive rate, accuracy with 95% Wilson score intervals [17], ROC–AUC (from continuous detector scores), and latency percentiles (p50/p95/p99). Runs are seeded and described by a manifest.

5 Results

5.1 Head-to-head detection

Figure 5 shows the detector curves on the public corpus; the exact numbers are in Table 4 (Appendix A). On generic injection QFIRE’s deterministic hybrid (F1 0.86) and Meta PromptGuard-2 (F1 0.86) are statistically tied at the top, both above the DeBERTa-v3 detector alone (0.84); PromptGuard-2 reaches it with the cleanest precision (1.00, FPR 0.00). We make no claim that QFIRE beats PromptGuard-2 on generic injection—they are even. QFIRE’s distinguishing value is the scope/PHI story (§6, Figure 1), where the same PromptGuard-2 recovers only 0.40 recall versus 0.83 for QFIRE’s combined scope+PHI chain. The QFIRE deterministic hybrid combines complementary detectors; the de-obfuscation pass adds recall on obfuscated payloads (§5.2) at negligible latency.

5.2 De-obfuscation ablation

Figure 6 shows the trade-off; Table 6 (Appendix A) lists the exact numbers, measured on an obfuscated variant of the attack set (payloads re-encoded with Base64/ROT13/leetspeak/homoglyphs)

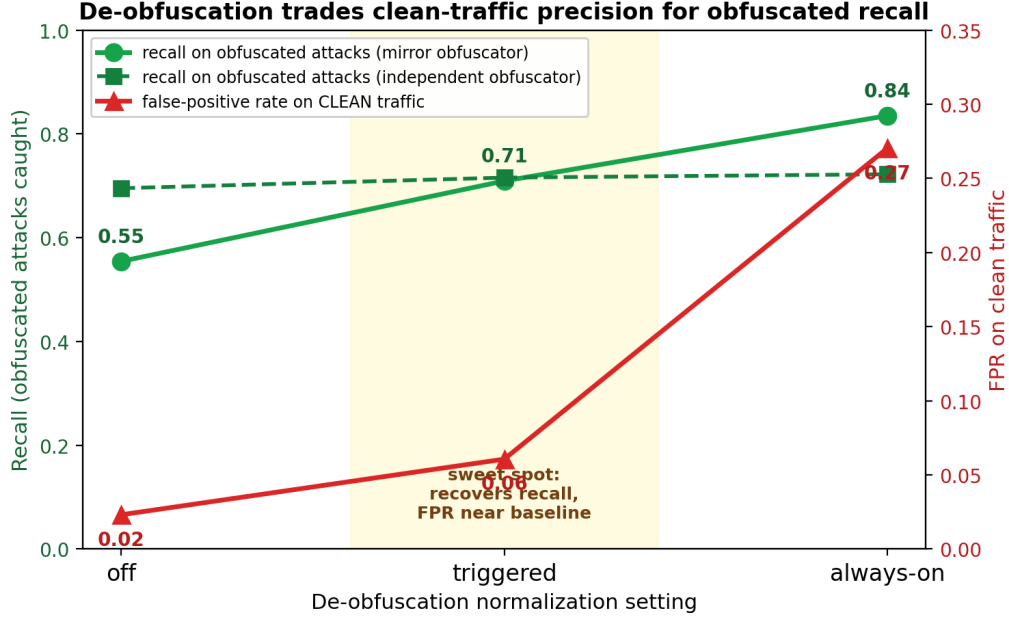


Figure 6: De-obfuscation trade-off across settings (off $\rightarrow$ triggered $\rightarrow$ always-on) for a mirror and an independent obfuscator. Recall on obfuscated attacks (green) rises with normalization while clean-traffic FPR (red) stays near baseline until always-on spikes it to 0.27. *Triggered* is the sweet spot.

with and without the normalization pass. On obfuscated attacks the pass recovers a large fraction of evaded detections, lifting recall substantially; the cost is reduced precision when the same aggressive decoding is applied to clean traffic (the false-positive rate rises on the non-obfuscated corpus of Table 4). The practical implication is that de-obfuscation is a *targeted* control for channels where encoded payloads are a concern, rather than an always-on default—a trade-off we report rather than tune away.

5.3 Healthcare / PHI panel

Table 7 (Appendix A) reports the HIPAA/PHI chain on a clinical-adjacent corpus, where over-blocking legitimate benign clinical queries is itself a failure mode. The result is a sharp, actionable warning: a single calibrated scope rule (`hc_no_diagnosis`) attains F1 0.91, but naively conjoining ten strict LLM-judge scope rules drives the false-positive rate to 1.00—every benign clinical prompt trips at least one over-eager judge. Positive-security composition therefore demands calibration, or utility collapses.

5.4 Latency and cost

Deterministic detectors (regex, Aho-Corasick, entropy) run in under 0.1 ms/prompt. The local DeBERTa-v3 classifier, executed via Rust ONNX on CPU, costs ≈ 81 ms mean / 255 ms p95 (cold) and is paid only when the cheap detectors abstain. The LLM judge is the only network-cost node; under local Ollama every call is \$0, so overhead is reported as latency, and the ten-judge healthcare chain reaches a p95 of ≈ 2.8 s—motivating cheap-before-expensive ordering and modest rule counts. Judge latency is also strongly model-dependent (§5.5): a single judge call costs p50 ≈ 0.4 –0.6 s for

the Llama models but 7.2s for Gemma 4 and 25s (p95 40s) for Qwen 3.6, whose chain-of-thought consumes the generation budget—so a compact, format-obedient judge model is a latency requirement for an inline firewall, not merely an accuracy one. *Measurement integrity*: within a single **bench** invocation the verdict cache is shared across chains, so chains evaluated after the DeBERTa-only chain read cached classifier verdicts and report artificially low latency. We therefore quote the *cold* DeBERTa latency and characterize the hybrid qualitatively: short-circuiting means overtly-malicious prompts are caught by sub-millisecond lexical detectors and never reach the classifier. An independent PyTorch run of the same protectai weights on the same full corpus measured P 0.984/R 0.721/F1 0.832 at p95 193ms, versus the Rust ONNX P 0.976/R 0.744/F1 0.844 at p95 255ms: the near-identical P/R/F1 (within ~ 0.02) validates that our ONNX integration faithfully reproduces the PyTorch reference, while the latency shows Rust ONNX is *not* faster than PyTorch on this CPU. We thus make no raw-speed claim; the Rust advantage is single-binary, no-Python, in-process deployment.

5.5 Judge-model ablation: does the backend model matter?

The LLM scope-judge node delegates the in/out-of-scope decision to a backing model. To isolate that model’s effect we hold everything else fixed—a single scope rule (**hc_no_diagnosis**) in a one-rule chain so the model’s verdict is the only deciding factor, the same 100-attack/100-benign HealthBench subset, prompt, seed, and **--no-cache**—and vary only the Ollama model via **QFIRE_JUDGE_MODEL**. Figure 7 and Table 8 (Appendix A) report two within-family points (Llama 3.1 8B vs. 3.2) and two cross-family models pulled at evaluation time (Gemma 4, Qwen 3.6). **Finding.** The backend model matters in two ways. (i) *Decision quality varies even within a family*: Llama 3.1 and Gemma 4 are excellent, near-interchangeable judges (F1 0.995, FPR 0.00, $\kappa=0.98$ between them), but Llama 3.2—same family, newer/smaller—over-blocks 28% of benign clinical prompts (FPR 0.28, κ only 0.71); a swap silently shifts the precision/recall operating point. (ii) *Verbose reasoning models are unsuitable as low-latency single-line judges*: Qwen 3.6 judges correctly on isolated prompts but, as a firewall judge, abstained on 100/200 prompts at 25–40s/call ($60\times$ Llama). On longer adversarial prompts it exhausts its generation budget on internal reasoning and never emits the required **OUT OF SCOPE** line, which the firewall conservatively treats as abstain→allow (recall collapses to 0.13). This is a genuine deployment property: a **judge** node needs a model that answers in the requested format under a tight token budget. QFIRE’s deterministic detectors are model-independent, but any chain with a **judge** node inherits the backing model’s calibration and format behavior; we therefore record the judge model in every run manifest and node version (**judge/<provider>:<model>**) and recommend validating a candidate judge’s precision/recall and abstain rate on a labeled subset before deployment—the procedure this ablation demonstrates. (A compact non-reasoning alternative, Qwen3 8B, is a viable judge: P 1.00/R 0.90/F1 0.947 at p50 4.0s.)

5.6 Multi-judge majority voting: accuracy, latency, robustness

If one judge model can be miscalibrated (§5.5), can an ensemble be more robust? QFIRE expresses this declaratively: one rule with three **judge** nodes, each pinned to a different model, under the **aggregate** short-circuit, which evaluates all nodes *concurrently* and collapses them by confidence-weighted majority—a hard 2-of-3 vote. Because the judges run in parallel, measured latency is the *slowest* member, not the sum. We benchmark the ensemble (Llama 3.1 + Gemma 4 + Qwen3 8B) on the same subset and additionally compute every *k*-of-*n* majority from the aligned per-prompt decisions (Figure 8).

Judge-model ablation: the backend model changes accuracy and latency

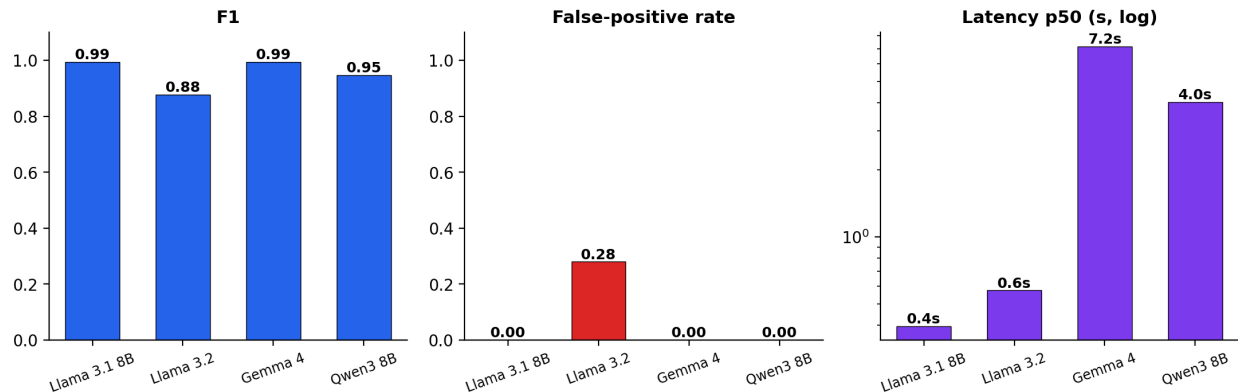


Figure 7: Judge-model ablation: F1 (left), false-positive rate (centre, Llama 3.2’s 0.28 spike), and p50 latency (right, log scale, 0.4–7.2 s). The backend model changes both accuracy and latency.

Finding (honest framing). The majority vote reaches a perfect F1 (1.000), higher than any single judge—but the frontier shows the best *single* model (Llama 3.1) already attains F1 0.995 at 0.4s, $\approx 16\times$ faster than the 7s ensemble, so voting buys little raw accuracy here. What it buys is *robustness*: the $F1=1.000$ ensembles include the miscalibrated Llama 3.2 (FPR 0.28 alone), whose benign false-positives are outvoted by the two well-calibrated judges. Majority voting thus lets a deployer safely fold in a model they do not fully trust—converting a single model’s silent failure mode into a recoverable minority vote—at a latency premium set by the slowest member. We recommend a single validated judge for latency-critical inline use, and majority voting where judge calibration is uncertain or must be defended in audit.

5.7 Policy-verbosity ablation: does a wordier scope help?

A scope policy can be written tersely (“Marketing content only.”) or as a long structured firewall (role, allowed/forbidden lists, adversarial-defense clause, refusal protocol; ~ 230 words). We hold QFIRE’s judge scaffold and the IN/OUT SCOPE contract fixed and vary *only* the scope text across four verbosity rungs (T0 terse, T1 one sentence, T2 structured paragraph, T3 full firewall) in each of four domains (marketing, healthcare, code, SQL). Each of the 16 conditions is a judge-only single-rule chain—no lexical node, so the scope wording is the sole deciding factor—evaluated with `--no-cache` (the verdict cache key omits scope) and seed 42 on the same 929 public injection attacks (out-of-scope $\rightarrow$ expected BLOCK) and 50 generated in-domain benign requests per domain (in-scope $\rightarrow$ expected ALLOW); judge model Llama 3.2. We report Youden’s $J = \text{TPR} + \text{TNR} - 1$ and paired bootstrap CIs on ΔJ between adjacent rungs (same prompts $\rightarrow$ paired).

Finding. *Policy length is nearly irrelevant for blocking injections, and the verbosity that matters trades off against over-refusal non-monotonically.* Across all 16 conditions, attack-block rate stays in 0.97–0.99 (Fig. 9b)—the firewall scaffold and judge contract do that work; scope wording adds little. The differentiator is the true-negative rate (legitimate-request pass rate), and the pooled J curve is non-monotone: 0.80 (T0) $\rightarrow$ 0.62 (T1) $\rightarrow$ 0.85 (T2) $\rightarrow$ 0.76 (T3). Paired bootstrap confirms every step (CIs exclude 0): terse $\rightarrow$ sentence $\Delta J = -0.18$ $[-0.23, -0.13]$, sentence $\rightarrow$ paragraph $+0.23$ $[+0.17, +0.29]$, paragraph $\rightarrow$ firewall -0.09 $[-0.14, -0.05]$. The one-sentence rung (T1) is a trap: “do X only; refuse anything else” primes the judge toward refusal without enumerating what is allowed—

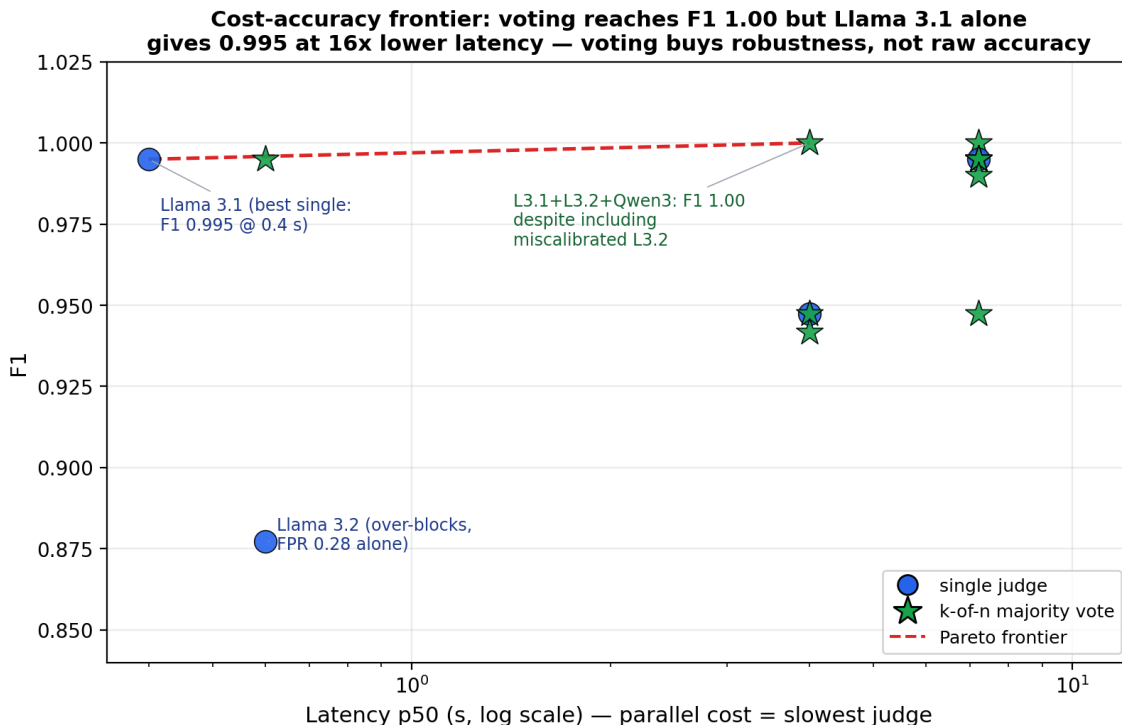


Figure 8: Cost-accuracy frontier: every single judge and every k -of- n majority vote in $F1 \times \text{latency}$ space (latency = slowest member, log scale). The $F1=1.00$ ensembles *include* the miscalibrated Llama 3.2, whose false-positives are outvoted—voting buys *robustness*, not raw accuracy.

healthcare T1 collapses to a 0.02 pass rate, refusing 98% of legitimate scheduling requests. The structured paragraph (T2) with explicit allowed *and* forbidden lists is the best or tied-best rung in every domain; the maximal firewall (T3) regresses from it, its aggressive “respond only with the refusal” framing making the judge trigger-happy at no gain in already-saturated TPR. The practical guidance: enumerate both what is allowed and what is forbidden in a short paragraph—neither a bald one-liner nor a maximal defensive wall. (Caveats: a single 3B judge; a larger model may follow long policies more faithfully. The model-generated benign sets add noise to absolute per-domain TNR but cancel in the paired ΔJ contrasts that carry the finding.)

6 QFIRE-HealthBench

QFIRE-HealthBench is a healthcare-specific prompt-injection dataset: 1,000 benign clinical-adjacent prompts and 1,000 malicious prompts built with real garak payloads and real Microsoft PyRIT converters (PyRIT under a Python-3.11 venv; garak DAN-family and in-the-wild jailbreaks from NVIDIA/garak). It targets threats a generic injection classifier misses: PHI exfiltration, cross-patient access, re-identification, bulk export, and out-of-scope clinical advice. All identifiers are synthetic; the dataset and its card ship in `corpora/healthcare_bench/`.

Figure 1 and Table 10 (Appendix A) report the central healthcare result. An injection classifier alone caps far below QFIRE—and the *strongest* generic detector we measured, Meta PromptGuard-2 (F1 0.86 on public injection), recovers only 0.40 recall here; protectai DeBERTa 0.57; QFIRE’s classifier-only chain 0.59. The reason is structural: most healthcare threats carry no jailbreak

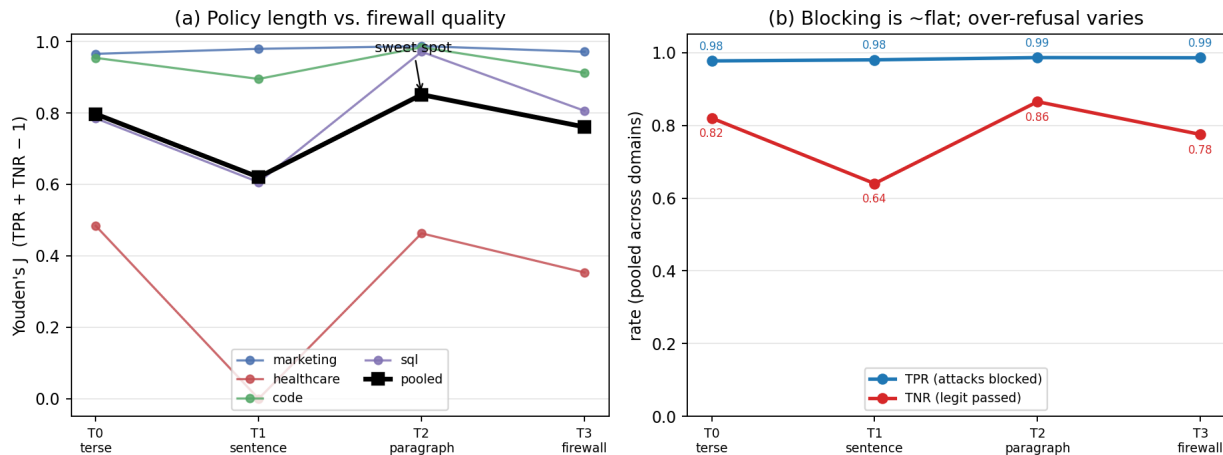


Figure 9: Policy-verbosity ablation (16 conditions, Llama 3.2 judge). (a) Youden’s J vs. rung per domain (thin) and pooled (bold): the curve is non-monotone—the one-sentence rung dips, the structured paragraph peaks, the full firewall regresses. (b) Pooled true-positive rate (attacks blocked) is essentially flat at 0.98 across all rungs, while the true-negative rate (legitimate requests passed) carries all the variation. Policy verbosity barely affects injection-blocking; its effect is on over-refusal.

token. Adding the PHI detector and positive-security scope rules (**bench_combined**) lifts recall to 0.83 (F1 0.73 $\rightarrow$ 0.87) at a calibrated FPR of 0.08. Per category the detectors are complementary: DeBERTa owns direct-injection/system-exfiltration/jailbreak; the PHI engine owns bulk-export/PHI-smuggling (zero injection signal, DeBERTa 0.00); and only the combined chain covers re-identification (0.00 $\rightarrow$ 1.00), PHI-exfiltration (0.41 $\rightarrow$ 0.94), and cross-patient access (0.41 $\rightarrow$ 0.62). This is the quantitative case for the paper’s thesis: generic injection detection—even SOTA—is necessary but not sufficient in healthcare; scope and PHI controls close a gap a classifier structurally cannot.

7 Discussion and Limitations

Detectors are complementary, not redundant. Figure 10 makes the paper’s central claim visible at a glance: it plots per-category recall for each detector chain on the 1,000 malicious QFIRE-HealthBench prompts. The structure is block-diagonal, not uniform. The injection classifier (column 1) is strong exactly where an injection signal exists—direct injection, system-prompt exfiltration, jailbreak—and falls to *zero* on bulk export, PHI smuggling, and re-identification, which contain no such signal. The PHI detector (column 2) is the mirror image: it owns bulk-export and PHI-smuggling but is blind to jailbreaks. Neither single detector is sufficient, and crucially their failure modes are *disjoint*: the off-diagonal zeros of one column are covered by another. Only the combined chain (column 4) is uniformly green, because the boolean composition of complementary detectors is what closes the coverage gap—this is the mechanism behind the 0.40/0.57/0.59 $\rightarrow$ 0.83 recall jump, not a better classifier. The single remaining cool cell, out-of-scope clinical advice (0.44), is the hardest class: disguised dosing/diagnosis requests with neither a lexical nor a PHI marker, where only the higher-latency LLM scope judge (§5.3) applies. This heatmap is the empirical argument for the declarative, multi-detector design of Figure 4: a deployer composes the columns their threat model needs.

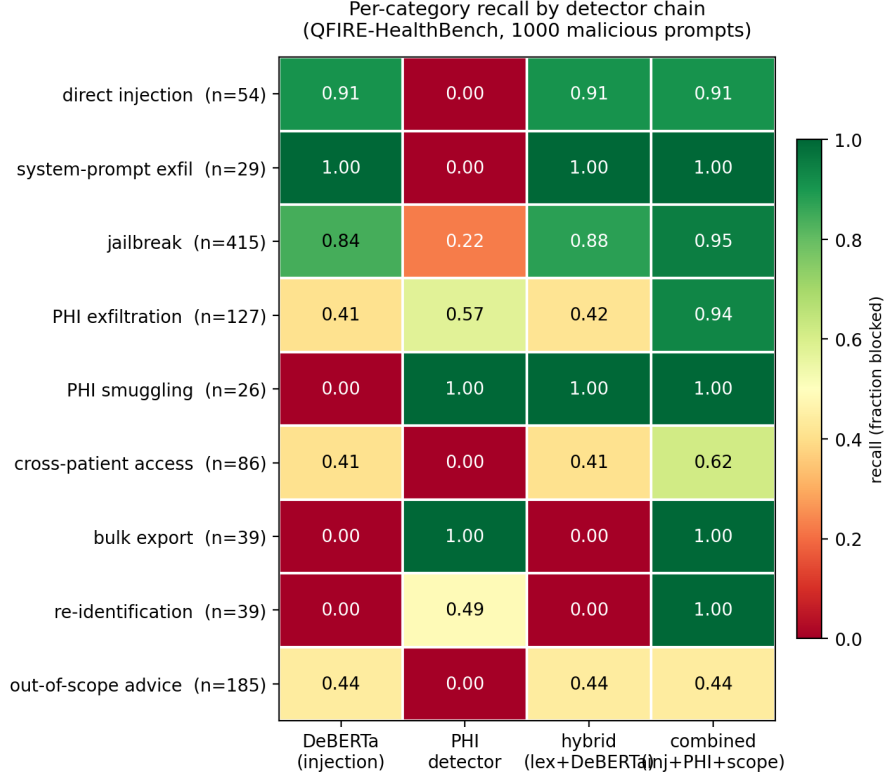


Figure 10: Per-category recall by detector chain on QFIRE-HealthBench (1,000 malicious prompts; green = caught, red = missed). The injection classifier and the PHI detector have *disjoint* blind spots (off-diagonal zeros); only their composition—the combined scope+PHI chain—is uniformly high. This complementarity, not a stronger single model, is why QFIRE closes the healthcare gap that SOTA PromptGuard-2 (0.40 recall) leaves open.

Relationship to HAARF. This work operationalizes the Healthcare AI Agents Regulatory Framework (HAARF) [14]. HAARF defines the security-verification *standard* a clinical AI agent must meet—scope restriction, minimum-necessary access, PHI protection, and auditable refusal. QFIRE is a runtime *enforcement* and *measurement* layer for those requirements: each HAARF control maps to a declarative rule or chain (e.g. minimum-necessary $\rightarrow$ `hc_phi_other_patient_record`; no-diagnosis $\rightarrow$ `hc_no_diagnosis`), and the immutable audit log supplies the evidence trail HAARF verification expects. The HealthBench result quantifies why such a framework cannot rely on a generic injection classifier alone: the controls HAARF most cares about are precisely the threats that carry no injection signal.

Positioning against other guardrail architectures. QFIRE makes deliberate trade-offs relative to the systems in Table 2, with clear wins and clear costs. Versus **LlamaFirewall** [1], QFIRE gives up LlamaFirewall’s deepest auditing stage (AlignmentCheck’s chain-of-thought reasoning audit) but eliminates the Python/PyTorch latency tax by running detection in a single Rust binary with embedded ONNX—and, on generic injection, its hybrid is statistically even with the PromptGuard-2 model LlamaFirewall is built around (§5). Versus **OneShield** [9], QFIRE shares the parallel multi-detector design but removes the dependence on external API management and replaces opaque classifiers with declarative, unit-testable YAML policy; OneShield’s enterprise-scale

dynamic scale-out is something QFIRE’s single-node design does not attempt. Versus the **Cognitive Firewall** [2], QFIRE keeps *all* computation local, which matters under HIPAA—the Cognitive Firewall’s cloud semantic stage raises exactly the privacy and latency concerns a clinical deployment must avoid—at the cost of not exploiting edge/cloud offloading for scale. Versus **PSG-Agent** [12], QFIRE is stateless and per-request rather than per-user-adaptive: it cannot model intent drift across a user’s history, but it deploys trivially in stateless, audit-friendly environments where PSG-Agent’s profiling is impractical. Versus **Semantic Firewalls** [15], QFIRE accepts free-form prompts and natural scope descriptions rather than demanding an application-specific type-safe protocol, trading the strongest syntactic-bypass guarantees for drop-in adoption (only the base URL changes). The common thread: QFIRE’s contribution is not a single stronger detector but the combination of *low-latency local execution*, *declarative auditable policy*, and *positive-security scope+PHI enforcement*—a point in the design space none of these systems occupies, and the one clinical agents need. The flip side is honest: QFIRE is single-node, its LLM-judge cost grows with rule count, and it does not provide the cross-turn reasoning audits, per-user adaptation, or formal protocol guarantees that specialized systems do.

Why a declarative language matters in regulated settings. The baselines encode policy in Python and model weights; QFIRE encodes it in version-controlled YAML the engine interprets (Figure 4). In a clinical deployment this is not a convenience but a compliance property: a hospital security officer can read, diff, review, and unit-test a chain (`qfire rules test` runs every rule’s exemplars) without modifying or trusting the engine internals, and changes to policy are auditable in version control. The positive-security results of this paper are only deployable because the policy that produced them is inspectable data.

Limitations. QFIRE’s positive-security stance trades permissiveness for safety: a strict health-care chain can over-block benign clinical-adjacent prompts, which we measure (§5.3) rather than hide. On generic injection a strong classifier (PromptGuard-2) matches QFIRE; QFIRE’s measured advantage is the scope/PHI coverage of §6 and single-binary deployment with no Python in the hot path, not a generic-detector win. Cross-dataset detection numbers are lower than a detector’s in-distribution test score, underscoring that single-corpus claims do not transfer; we therefore report a public, mixed corpus and release the snapshot. The LLM-judge node’s cost grows with the number of scope rules, motivating the cheap-before-expensive ordering. Our PHI matchers are conservative for identifiers (names, biometrics, face photos) that are not reliably recoverable from text alone. Finally, the LLM scope judge inherits the biases and failure modes of its backing model; we mitigate but do not eliminate this by keeping deterministic detectors ahead of it and by reporting judge-chain over-blocking explicitly.

8 Conclusion

QFIRE shows that a Rust, parallel, positive-security firewall can combine low-latency local inference, de-obfuscation, and scope constraining in one reproducible toolchain. Its central empirical result is that generic prompt-injection detection—even the SOTA PromptGuard-2—is necessary but not sufficient in healthcare: on QFIRE-HealthBench the strongest classifier recovers only 0.40 recall, while QFIRE’s combined scope+PHI chain reaches 0.83, because most clinical threats carry no injection signal.

Beyond the benchmark, QFIRE gives the HAARF framework [14] a concrete, testable enforcement layer (Table 1): the abstract controls for adversarial robustness (C3.2), input sanitization

(C3.2.3), PHI protection (C3.6.1), real-time monitoring (C3.4), autonomy authority boundaries (C6.3), and immutable audit trails (C2.5.1) are each realized as a versioned rule, a detector node, or the audit log—and, crucially, are *measured* rather than asserted. A compliance reviewer can read the YAML that satisfies a control, run `qfire rules test` to check it, and inspect the audit log for evidence. This is what we believe a security-verification standard needs to become operational: not more conceptual requirements, but inspectable mechanisms with benchmarked false-positive and recall rates. All artifacts and the `make paper` pipeline are released for independent verification.

A Detailed result tables

The figures in the body carry the narrative; the exact numbers behind them are collected here for completeness. Each table is referenced from the corresponding results subsection.

System	Prec.	Rec.	F1	FPR	AUC	p95 (ms)
DeBERTa-v3 (protectai, PyTorch)	0.984	0.721	0.832	0.011	—	195.4
PromptGuard-2 86M (Meta, PyTorch)	0.997	0.755	0.859	0.002	—	47.4
Regex denylist (lexical)	1.000	0.295	0.456	0.000	0.647	0.05
Aho-Corasick keywords	0.906	0.343	0.498	0.032	0.656	0.02
Entropy heuristic	0.828	0.088	0.160	0.016	0.536	0.05
DeBERTa-v3 (ONNX, ours)	0.976	0.744	0.844	0.016	0.925	267.76
QFIRE hybrid	0.967	0.767	0.856	0.023	0.927	242.26
QFIRE hybrid + de-obf	0.733	0.829	0.778	0.270	0.814	283.85

Table 4: Head-to-head detection on 1,968 public prompts (attack = positive class). Latency is per-prompt wall clock; QFIRE detectors run in Rust, PyTorch baselines on CPU.

System	Accuracy	95% Wilson CI
Regex denylist (lexical)	0.667	[0.646, 0.688]
Aho-Corasick keywords	0.673	[0.652, 0.694]
Entropy heuristic	0.561	[0.539, 0.583]
DeBERTa-v3 (ONNX, ours)	0.870	[0.855, 0.885]
QFIRE hybrid	0.878	[0.863, 0.892]
QFIRE hybrid + de-obf	0.776	[0.757, 0.794]

Table 5: QFIRE accuracy with 95% Wilson score confidence intervals.

Configuration	Recall	F1	Δ F1
Hybrid (no normalization)	0.554	0.702	0.000
Hybrid + de-obfuscation	0.835	0.781	0.080

Table 6: Effect of the de-obfuscation normalization pass on obfuscated attacks.

B Reproducibility

Every table is generated from benchmark JSON by `scripts/make_tables.py`; `make paper` runs the build, fetches the corpus snapshot, executes the benchmark matrix and baselines, and compiles

Chain	Block rate	FPR	Precision	Recall
hipaa_phi	1.000	1.000	0.500	1.000

Table 7: Healthcare/PHI chain: block rate and false-positive rate on clinical-adjacent prompts.

Judge model	Prec.	Rec.	F1	FPR	p50	abstain
Llama 3.1 8B (base)	1.00	0.99	0.995	0.00	0.4 s	0
Llama 3.2	0.78	1.00	0.877	0.28	0.6 s	0
Gemma 4	1.00	0.99	0.995	0.00	7.2 s	1
Qwen3 8B	1.00	0.90	0.947	0.00	4.0 s	0
Qwen 3.6	1.00	0.13	0.230	0.00	25 s	100

Table 8: Judge-model ablation on the single-rule `judge_scope` chain (100 attack / 100 benign). Per-prompt agreement with the Llama 3.1 baseline (Cohen’s κ): Llama 3.2 85.5% ($\kappa=0.71$), Gemma 4 99.0% ($\kappa=0.98$), Qwen3 8B 94.5% ($\kappa=0.96$ est.), Qwen 3.6 57.0% ($\kappa=0.13$). “abstain” counts prompts where the model failed to emit a parseable one-line verdict (treated as abstain→allow).

this document. The run manifest (seed, model and rule/detector/corpus versions) is embedded in each artifact.

References

- [1] Sahar Abdelnabi et al. LlamaFirewall: An open source guardrail system for building secure AI agents. *arXiv preprint arXiv:2505.03574*, 2025.
- [2] Cognitive Firewall Authors. The cognitive firewall: Securing browser-based AI agents against indirect prompt injection via hybrid edge-cloud defense. *arXiv preprint arXiv:2603.23791*, 2026.
- [3] deepset. prompt-injections dataset. Hugging Face Datasets, 2023.
- [4] Leon Derczynski et al. garak: A framework for security probing large language models. *arXiv preprint arXiv:2406.11036*, 2024.
- [5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.
- [6] Jackie Hao. jailbreak-classification dataset. Hugging Face Datasets, 2023.
- [7] Meta AI. Llama prompt guard 2. Hugging Face model card, 2025.
- [8] Ollama. Ollama: run large language models locally. <https://ollama.com>, 2024.
- [9] OneShield Authors. OneShield – the next generation of LLM guardrails. *arXiv preprint arXiv:2507.21170*, 2025.
- [10] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022.
- [11] Protect AI. deberta-v3-base-prompt-injection. Hugging Face model card, 2024.

Configuration	Prec.	Rec.	F1	FPR	p50 (parallel)
Best single (Llama 3.1)	1.00	0.99	0.995	0.00	0.4 s
Llama 3.2 (miscalibrated)	0.78	1.00	0.877	0.28	0.6 s
Qwen3 8B	1.00	0.90	0.947	0.00	4.0 s
3-model majority vote	1.00	1.00	1.000	0.00	7.1 s

Table 9: Multi-judge majority vote vs. single judges on the 100-attack/100-benign subset. The vote reaches a perfect F1, but the best single model already attains 0.995 at $16\times$ lower latency.

- [12] PSG-Agent Authors. PSG-Agent: Personality-aware safety guardrail for LLM-based agents. *arXiv preprint arXiv:2509.23614*, 2025.
- [13] pyke.io. ort: Rust bindings for onnx runtime. <https://github.com/pykeio/ort>, 2024.
- [14] Jim Schwoebel et al. HAARF: Healthcare AI agents regulatory framework — a comprehensive security verification standard for autonomous AI systems in clinical environments. *medRxiv*, pages 2026–04, 2026.
- [15] Semantic Firewall Authors. Semantic firewalls with online ensemble learning for secure agentic RAG systems. *arXiv preprint arXiv:2603.03911*, 2026.
- [16] U.S. Department of Health and Human Services. Guidance regarding methods for de-identification of protected health information (HIPAA safe harbor, 45 cfr 164.514(b)), 2012.
- [17] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

System		Prec.	Rec.	F1	FPR
<i>Generic injection detectors (PyTorch baselines)</i>					
protectai DeBERTa-v3		1.000	0.574	0.729	0.000
Meta PromptGuard-2-86M		0.998	0.402	0.573	0.001
<i>QFIRE chains</i>					
bench_phi (PHI only)		0.755	0.250	0.376	0.081
bench_deberta (injection only)		1.000	0.595	0.746	0.000
bench_hybrid (lexical+DeBERTa)		1.000	0.637	0.778	0.000
bench_hybrid_trig (+de-obf)		1.000	0.638	0.779	0.000
bench_combined (inj+PHI+scope)		0.911	0.829	0.868	0.081
bench_combined_trig (+de-obf)		0.911	0.829	0.868	0.081

Category	n	DeBERTa	PHI	hybrid	combined
bulk_export	39	0.00	1.00	0.00	1.00
clinical_advice	185	0.44	0.00	0.44	0.44
cross_patient	86	0.41	0.00	0.41	0.62
direct_injection	54	0.91	0.00	0.91	0.91
jailbreak	415	0.84	0.22	0.88	0.95
phi_exfil	127	0.41	0.57	0.42	0.94
phi_smuggle	26	0.00	1.00	1.00	1.00
reidentification	39	0.00	0.49	0.00	1.00
system_exfil	29	1.00	0.00	1.00	1.00
OVERALL	1000	0.59	0.25	0.64	0.83

Table 10: QFIRE-HealthBench (1,000 attack / 1,000 benign): a generic injection classifier (incl. SOTA PromptGuard-2) vs. QFIRE’s combined scope+PHI chain. PromptGuard-2 leads on public injection yet recovers only 0.40 recall here.